UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIA E TECNOLOGIA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO
DCC606 – ANÁLISE DE ALGORITMOS – 2026
PROF. DR. HEBERT OLIVEIRA ROCHA

HELIAN VINICIUS FILINTO DA SILVA
JOSÉ CARVALHO NETO

ANÁLISE DO ALGORITMO DA TORRE DE HANÓI

Estudo Comparativo de Eficiência e Complexidade

BOA VISTA – RR

ABRIL DE 2026

Sumário

1 Introdução **3**
1.1 O que é a Torre de Hanói? 3
1.2 Descrição do Problema 3
1.3 Regras do Jogo 3

2 Objetivos **4**

3 Metodologia **4**

4 Análise Teórica: Dedução por Expansão **4**

5 Prova por Indução Matemática **4**
5.1 Base da Indução 5
5.2 Hipótese de Indução 5
5.3 Passo Indutivo 5

6 Desenvolvimento dos Algoritmos **5**
6.1 Algoritmos Implementados 5

7 Conclusão **6**
7.1 Discussão dos Resultados 6
7.2 Considerações Finais 6

Referências **8**

1 Introdução

Este relatório foi desenvolvido como parte da atividade de seminário da disciplina Análise de Algoritmos, ministrada pelo Prof. Dr. Hebert Oliveira Rocha, na Universidade Federal de Roraima (UFRR). O trabalho tem como objetivo apresentar um estudo experimental sobre o algoritmo da Torre de Hanói, um problema clássico amplamente utilizado no ensino de recursão e análise de complexidade computacional.

O foco principal da análise está no desempenho computacional do algoritmo, considerando diferentes implementações. Busca-se investigar como decisões de programação e estratégias de implementação podem influenciar diretamente o tempo de execução, mesmo quando o problema resolvido permanece o mesmo.

Para atingir esse objetivo, foram realizados testes controlados e repetidos, possibilitando a coleta sistemática de dados de execução. Os resultados obtidos são organizados e analisados ao longo do relatório, servindo de base para comparações e discussões que contribuem para a compreensão prática do impacto da implementação na eficiência de algoritmos.

1.1 O que é a Torre de Hanói?

A Torre de Hanói é um problema clássico da matemática e da computação, amplamente utilizado no estudo de recursão, estruturas de dados e análise de algoritmos. Ele é frequentemente empregado como exemplo didático para compreender chamadas recursivas e o crescimento exponencial do custo computacional.

1.2 Descrição do Problema

O problema consiste em mover uma pilha de discos de uma haste de origem para uma haste de destino, utilizando uma haste auxiliar, obedecendo a um conjunto de regras bem definidas. Inicialmente, todos os discos estão empilhados na haste de origem, organizados em ordem decrescente de tamanho, com o maior disco na base e o menor no topo.

1.3 Regras do Jogo

Durante a execução do problema, devem ser respeitadas as seguintes regras:

- Apenas um disco pode ser movido por vez;
- Um disco maior nunca pode ser colocado sobre um disco menor;
- Todos os discos devem começar empilhados corretamente na haste de origem.

2 Objetivos

Este trabalho visa comparar o desempenho prático e a complexidade teórica das implementações recursiva e iterativa. Busca-se validar a natureza exponencial do algoritmo e observar o impacto do gerenciamento de hardware no tempo de processamento.

3 Metodologia

Foram implementadas as versões recursiva e iterativa em C. Os testes utilizaram a API `QueryPerformanceCounter` no Windows para medições em microssegundos (μs), com 20 repetições para cada valor de n.

4 Análise Teórica: Dedução por Expansão

A relação de recorrência fundamental é:

$$T(n) = 2T(n-1) + 1 \tag{1}$$

Pelo método da expansão (iteração):

$$\begin{aligned}
T(n) &= 2T(n-1) + 1 \\
T(n) &= 2[2T(n-2) + 1] + 1 = 2^2T(n-2) + 2 + 1 \\
T(n) &= 2^3T(n-3) + 2^2 + 2^1 + 2^0
\end{aligned}$$

Para $k = n$, e considerando o caso base $T(0) = 0$, obtemos a série geométrica:

$$T(n) = \sum_{i=0}^{n-1} 2^i = \frac{1(2^n - 1)}{2 - 1} = 2^n - 1 \tag{2}$$

5 Prova por Indução Matemática

A indução matemática é um método de prova fundamental para demonstrar a validade de enunciados para todos os números naturais. O processo assemelha-se a uma sequência de dominós: provamos que a primeira peça cai (Base) e que, se uma peça cai, a próxima também cairá (Passo Indutivo).

Nesta seção, provamos que a forma fechada $J(n) = 2^n - 1$ é a solução exata para a recorrência $T(n) = 2T(n-1) + 1$.

5.1 Base da Indução

Provamos que o enunciado vale para os casos iniciais:

- **Para $n = 1$:** $J(1) = 2^1 - 1 = 1$. Como $T(1) = 1$, a base é válida.
- **Para $n = 2$:** $J(2) = 2^2 - 1 = 3$. Como $T(2) = 3$, o enunciado se mantém.

5.2 Hipótese de Indução

Supomos que a fórmula é verdadeira para um elemento genérico $n - 1$:

$$T(n-1) = J(n-1) = 2^{n-1} - 1 \tag{3}$$

5.3 Passo Indutivo

Objetivamos provar que $T(n) = J(n) = 2^n - 1$. Partindo da definição da recorrência e aplicando a hipótese de indução:

$$\begin{aligned}
T(n) &= 2 \cdot T(n-1) + 1 \\
T(n) &= 2 \cdot J(n-1) + 1 \\
T(n) &= 2 \cdot (2^{n-1} - 1) + 1 \\
T(n) &= 2 \cdot 2^{n-1} - 2 + 1 \\
T(n) &= 2^1 \cdot 2^{n-1} - 1 \\
T(n) &= 2^{1+n-1} - 1 \\
T(n) &= 2^n - 1
\end{aligned}$$

6 Desenvolvimento dos Algoritmos

Nesta etapa, descreve-se a lógica e as motivações por trás das duas abordagens utilizadas para resolver o problema, destacando como a implementação impacta o uso dos recursos de hardware.

6.1 Algoritmos Implementados

- **Torre de Hanói Recursiva:** Implementação baseada no paradigma de "Dividir e Conquistar", seguindo estritamente a recorrência matemática do problema. Embora seja o código mais elegante e fácil de ler, ela é considerada a "opção lenta"devido ao *overhead* constante de chamadas de função.

Cada movimento exige que o processador manipule a pilha de execução (*stack*), salvando contextos de memória bilhões de vezes, o que gera um custo computacional adicional perceptível em grandes valores de n.

- **Torre de Hanói Iterativa:** Versão otimizada que substitui as chamadas recursivas por laços de repetição (*loops*) e lógica binária/*bitwise*. Esta implementação é a "opção rápida"porque trabalha de forma contínua, evitando o peso da pilha do sistema.

  Mesmo mantendo a mesma complexidade teórica, ela demonstra uma eficiência de hardware superior, resultando em tempos de execução significativamente menores e maior estabilidade térmica para o processador.

7 Conclusão

A análise comparativa entre as implementações recursiva e iterativa da Torre de Hanói permite concluir que, embora ambos os algoritmos compartilhem a mesma complexidade assintótica exponencial $O(2^n)$, os resultados práticos revelam disparidades significativas de desempenho. O estudo evidencia que, enquanto a complexidade teórica define o comportamento de crescimento do algoritmo, o tempo real de execução é fortemente condicionado pela forma como o código interage com os recursos de hardware.

7.1 Discussão dos Resultados

A divergência observada nos tempos de processamento fundamenta-se na mecânica de execução de cada abordagem. A versão recursiva, fundamentada na fragmentação do problema em instâncias menores, impõe ao sistema a necessidade de organizar e gerenciar uma pilha de chamadas (*call stack*) que cresce proporcionalmente ao número de discos. Essa organização extra gera um *overhead* que se traduz em perda de eficiência em larga escala.

Em contrapartida, a implementação iterativa opera de forma direta. Ao utilizar laços de repetição e lógica *bitwise*, o algoritmo elimina o empilhamento de contextos de memória, reduzindo drasticamente a sobrecarga interna do processador.

7.2 Considerações Finais

Dessa forma, os dados coletados reforçam que a diferença de performance observada nos gráficos de tempo não está ligada à quantidade total de movimentos necessários — que permanece constante em ambos os casos — mas sim à eficiência do processamento desses movimentos pelo hardware.

Este trabalho reafirma a importância vital de analisar o desenvolvimento de algoritmos sob uma perspectiva dual: a solidez da teoria da complexidade aliada à otimização prática da implementação. A escolha de uma estratégia iterativa demonstra-se, portanto, como a solução mais robusta para sistemas que exigem alta performance e menor consumo de recursos computacionais.

Referências

KHAN ACADEMY. Torres de Hanói (artigo) | Algoritmos. Disponível em: https://pt.khanacademy.org/computing/computer-science/algorithms/towers-of-hanoi/a/towers-of-hanoi.

MAIOLI, Douglas. Torre de Hanói - Aula 01 - Regras, Como resolver e Fórmula da quantidade de movimentos. Disponível em: https://www.youtube.com/watch?v=Q2BooYpqS6g. Acesso em: 23 abr. 2026.

MAIOLI, Douglas. Torre de Hanói - Aula 02 - Demonstração por indução da fórmula da quantidade de movimentos. Disponível em: https://www.youtube.com/watch?v=ieaHP5_0QiA. Acesso em: 23 abr. 2026.

CORMEN, T. H. et al. Introduction to Algorithms. 3. ed. [s.l.]: MIT Press, 2009.